

Numpydantic: Static and dynamic array constraint validation for data standards and working programmers

Jonny L. Saunders^{1,2} and Daniel Aharoni¹

¹ University of California, Los Angeles. Department of Neurology. ² Peertech Global Cyberindustrial Concern LLC, LLC.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Johan Larsson

Reviewers:

- @niksirbi
- @kirangadhav
- @JoeZiminski

Submitted: 11 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Numpydantic is a minimalistic Python package that provides abstract types and a syntax for specifying constrained n-dimensional arrays. Its constraints can be validated both dynamically at runtime and statically using mypy. Rather than being bound to a single array framework, numpydantic is a generic array specification with an extensible interface system that currently supports numpy, hdf5, zarr, and video files. Designed to expose arrays to pydantic's validation framework, its NDArray type allows researchers to compose multiple arrays into validating data models, including generation of JSON schema, roundtrip serialization, and lazy proxies for on-disk arrays. A mypy plugin extends the quality of life improvements from static type analysis to the world of array programming. Numpydantic is part of a broader effort to lower barriers to structured data and provide a path to true cross-standard interoperability: decoupling schemas, implementations, and serialization formats for formal data standards, and providing simple tools responsive to the needs of everyday research programming.

Statement of need

Arrays are rarely formless bundles of numbers, they are "videos," "time series," "training and testing data," etc. where attributes like orientation, shape, and dtype are an intrinsic part of their definition as a type. Accordingly, outside of array frameworks themselves, array code rarely accepts arbitrary arrays. In Python this information lives only in docstrings, is invisible to type checkers, and requires large amounts of repetitive boilerplate to validate.

The lofty ambitions of open data require standards and formats (Wilkinson et al., 2016). Scientific data standards must specify array constraints, but the implementation of those constraints often binds the abstract schema to a single array framework and serialization format. Researchers have a rich set of static analysis tools to assist with programming, but still struggle with the hundreds of intermediate arrays that flow through their code.

This tension creates two unhappy local minima: on one side, data standards are often brittle and difficult to integrate into the code used for real-world analysis, relegated to a conversion stage after the work is done that can be lossy and labor-intensive. On the other, labs run on ad-hoc, local data structures that make even intra-lab collaboration difficult.

In the middle is a gap in tooling that responds to both categories of needs:

- Strict, validating, metadata enriched schemas with loose coupling to storage formats and programming interfaces
- Simple, low-overhead structure that can embed in existing code and support researcher-developer experience.

State of the field

Python's type annotation system has evolved dramatically since its introduction in 2014 (Python Software Foundation, n.d.; van Rossum et al., 2014). Typing arrays is an ongoing challenge, motivating type system features like variadic generics for shape specification (Mendoza & Pradeep Kumar Srinivasan, 2020). Numpy (Harris et al., 2020) introduced type annotations for its arrays in 2021 (NumPy 1.20.0 Release Notes, 2021), with steady improvements over time¹. Numpy's types allow specifying shape and dtype, but not extended expressions like explicit dimension sizes, size ranges, etc. Numpydantic extends previous work on runtime-checkable types, vendoring nptyping (Hagenaars, 2019)² and extending and updating its syntax.

Other array frameworks like Dask (Dask Development Team, 2016) and Zarr (Miles et al., 2020) are either untyped or allow typing only for dtype.

Dataclasses (Smith, 2017) use type annotations to create intelligible data structures, but the annotations are unvalidated and used solely in static analysis. Pydantic (Colvin et al., 2026) is the most widely used library in Python for declaring runtime-validating dataclass-like models, but it lacks builtin support for arrays.

The closest sibling to numpydantic is likely pandera (Bantilan, 2020), which provides schemas for dataframes across multiple backends (and, recently, complex multi-arrays from xarray). We are not aware of other projects that provide array specifications and validation across multiple array backends³.

Software design

Numpydantic is designed to be a dependency you don't have to think about - it has two required dependencies, numpy and pydantic, which are already likely present in any package that wants to use it. It can be used with or without pydantic: its annotations can be used in the typing layer without modifying existing runtime code.

Support for multiple array frameworks is implemented as a set of interface classes, where each interface overrides the methods needed to match input to an interface, extract shape and dtype, serialize and deserialize json, among other features.

¹See also the predecessor numpy-stubs (numpy, 2017) and the experimental numtype (numpy, 2025).

²As of writing, nptyping is unmaintained, without update since February 2023.

³If you maintain an array spec library for Python, please contact us and we will link to you from our docs!

67 Pydantic Models

68 At its simplest, numpydantic does this:

```
import numpy as np
from pydantic import BaseModel
from numpydantic import NDArray, Shape

class MyModel(BaseModel):
    array: NDArray[Shape["* t, * x, * y, 3 rgb"], np.uint8]

# numpy
MyModel(array=np.zeros((100, 1080, 1920, 3), dtype=np.uint8))
# dask
MyModel(array=dask.array.zeros((100, 1080, 1920, 3), dtype=np.uint8))
# hdf5
MyModel(array=('data.h5', '/nested/dataset'))
# zarr
MyModel(array=zarr.zeros((100, 1080, 1920, 3), dtype=np.uint8))
MyModel(array=('data.zarr', '/nested/dataset'))
# video
MyModel(array="data.mp4")
```

69 When used in pydantic models, numpydantic provides runtime validation for array specifications,
70 as well as access to on-disk arrays by reference.

71 By default, an array from any supported array backend can be used (as long as it matches the
72 specification), but an explicit array backend can be specified using an annotated type

```
class MyModel(BaseModel):
    array: Annotated[np.ndarray, NDArraySchema(Shape(1, 2, 3), np.uint8)]
```

73 Numpydantic constructs a valid JSON schema for its annotations, making them portable across
74 languages, and usable to accept array data via web APIs via, e.g. [fastAPI](#).

75 This simple example:

```
class MyModel(BaseModel):
    array: NDArray[Shape[2, "2-5"], np.float16]
MyModel.model_json_schema()
```

76 produces JSON Schema⁴:

```
{ "properties": { "array": {
  "dtype": "numpy.float16",
  "items": {
    "items": {
      "minimum": -65504.0, "maximum": 65504.0,
      "type": "integer"
    },
    "minItems": 2, "maxItems": 5,
    "type": "array"
  },
  "minItems": 2, "maxItems": 2,
  "type": "array"
}}
```

⁴All examples are abbreviated and formatted for clarity.

77 Numpydantic supports roundtripping array data to and from JSON, including any metadata
78 needed to reconstruct an array, with on-disk arrays optionally dumped as a reference rather
79 than loading and serializing the whole array.

80 e.g. for a dataset with scalar metadata alongside array data split between in-memory and
81 on-disk arrays, as would be common in both data standards and unstructured lab code:

```
MyDataset(
    name = "My Dataset",
    time = datetime.now(UTC),
    scores = np.zeros((2, 2)),
    raw = ZarrArrayPath("./data.zarr", "/dataset"),
    video = Path('./video.mp4')
).model_dump_json(round_trip=True)
```

82 A lightweight metadata descriptor can be dumped to JSON and reloaded later:

```
{ "name": "My Dataset",
  "time": "2026-06-11T02:17:53.973174Z",
  "scores": {
    "type": "numpy", "dtype": "float64", "shape": [ 2, 2 ],
    "value": [ [ 0.0, 0.0 ], [ 0.0, 0.0 ] ]
  },
  "raw": {
    "type": "zarr", "dtype": "float64",
    "file": "data.zarr", "path": "/dataset",
    "info": {
      "Shape": "(2000, 2000)",
      "hexdigest": "86c96d94e76c3b0f210f5a0bf2d8811d1ffd883d"
    }
  },
  "video": { "type": "video", "file": "video.mp4" }
}
```

83 Standalone Use

84 Numpydantic can be used without pydantic to annotate normal functions and classes!

```
def some_analysis(array: NDArray[Shape[1, 2, 3]]) -> NDArray[Shape[2, 4, 6]]:
    ...
```

85 These annotations can also be validated at runtime without pydantic using a tool like [beartype](#)
86 ([Curry, 2020](#)).

87 NDArray can also be used as a runtime type for instance checking and ad-hoc validation

```
>>> Grayscale = NDArray[Shape["*", "*"], np.uint8]
>>> array = np.zeros((100, 100), np.uint8)
>>> isinstance(np.zeros((100, 100), np.uint8), Grayscale)
True
>>> isinstance(np.zeros((100,)), Grayscale)
False
>>> Grayscale(np.zeros((100,)), np.uint8)
ShapeError: Invalid shape! expected shape ['*', '*'], got shape (100,)
>>> Grayscale(np.zeros((100, 100), np.float64))
DTypeError: Invalid dtype! expected <class 'numpy.uint8'>, got float64
```

88 Mypy plugin

89 A mypy plugin supports static analysis of numpydantic's specification syntax, enriching array
90 constructors with shape and dtype information for any supported array backend.

```
reveal_type(np.zeros((3, 4), dtype=np.uint8))
reveal_type(zarr.zeros((3, 4), dtype=np.uint8))
```

91 Without the plugin:

92 Revealed type is "numpy.ndarray[tuple[int, int], numpy.uint8]"
93 Revealed type is "Any"

94 With the plugin:

95 Revealed type is "numpy.ndarray[tuple[Literal[3], Literal[4]], numpy.uint8]"
96 Revealed type is "numpy.ndarray[tuple[Literal[3], Literal[4]], numpy.uint8]"

97 By annotating analysis and i/o functions with type information, researchers can avoid tedious
98 bugs from misshaped or mistyped arrays, and use the runtime types for reusable, minimal
99 boilerplate validation.

```
def analyze_grayscale(frame: NDArray[Shape[1080, 1920], np.uint8]):
    # ...
```

```
analyze_grayscale(np.zeros((1080, 1920, 3), dtype=np.uint8))
```

```
100 error: Argument "frame" to "analyze_grayscale" has incompatible type
101     "ndarray[tuple[Literal[1080], Literal[1920], Literal[3]], numpy.uint8]";
102     expected "ndarray[tuple[Literal[1080], Literal[1920]], np.uint8]"
```

103 Research impact statement

104 Numpydantic was developed as part of [LinkML arrays](#) (Moxon et al., 2026). Briefly, LinkML is
105 an accessible, expressive schema language that allows a schema defined once to be translated
106 into a number of schema and programming languages like OWL, JSON Schema, SQL DDL,
107 etc. LinkML is widely used for linked data schemas and ontologies, and is increasingly an
108 adoption target for scientific data formats. Support for n-dimensional arrays is a critical part
109 of using LinkML for scientific data, and numpydantic is the core array implementation for
110 LinkML in Python.

111 Numpydantic was also developed as part of an effort to rearchitect Neurodata Without Borders
112 (NWB) (Rübel et al., 2022), the de-facto data standard for systems neuroscience ([nwb-linkml](#)).
113 NWB uses HDMF (Tritt et al., 2019) as its schema language, which was developed out of
114 necessity as most schema languages do not support arrays. HDMF and NWB are both tightly
115 coupled to HDF5, which makes indexing and partial use of datasets extraordinarily costly
116 - to read a scalar metadata attribute, one needs to download the entire dataset, which for
117 neuroscientific data often means hundreds of gigabytes⁵. Backing NWB with linkml arrays and
118 numpydantic allows array backends to be freely swapped, and separates metadata from heavier
119 raw array data. By decoupling schema, programmatic APIs, and serialization formats with
120 LinkML as a *lingua franca*, we are working towards a true ecosystem of FAIR data (Wilkinson
121 et al., 2016) where data standards only need to define the schema layer and get the rest for
122 free.

⁵See also [LINDI](#) (Neurodata Without Borders, 2024), an overlay for addressing streaming data needs with HTTP range requests.

Limitations & Future Directions

- We are working on [v2.0 of numpydantic](#), where we will replace the string-based syntax borrowed from `nptyping` with modern Python typing features.
- Until then, non-mypy static type checkers have trouble checking some forms of `NDArray` types. There are [several intermediate solutions](#), like specifying shape arguments as literals, and the `NDArraySchema` class.
- We will be extending the specification syntax to include more advanced array features, like chunking and memory order.
- Support for `TypeVars` that can indicate that a given axis can be any size, but all axes within a given scope must be the same size is important for many scientific applications (e.g. a timestamps array must be the same length as a video). We are [actively working on this](#) in our mypy plugin and [upstream in mypy itself](#)

AI usage disclosure

No “AI” or LLMs were used at any time during development, nor to produce any part of this submission.

Acknowledgements

Thanks to Ryan Ly, Chris Mungall, Oliver Rübel, Wouter-Michiel Vierdag, Ben Dichter, and John-Marc Chandonia for co-designing the LinkML Arrays specification which gave shape to the initial spec; to Antônio Camargo for setting up the conda forge feedstock; to all collaborators past and present; and to Rumbly Tumbly Lawnmower for being the light of my life and keeping my lap warm as I worked.

We are grateful for the funding that supported this work: NIH Director’s New Innovator Award, DP2MH129986.

References

- Bantilan, N. (2020). *Pandera: Statistical Data Validation of Pandas Dataframes*. 116–124. <https://doi.org/10.25080/Majora-342d178e-010>
- Colvin, S., Jolibois, E., Ramenzani, H., Garcia Badaracco, A., Dorsey, T., Montague, D., Matveenko, S., Trylesinski, M., Runkle, S., Hewitt, D., Hall, A., & Plot, V. (2026). *Pydantic Validation* (Version 2.14.0). <https://pydantic.dev/docs/validation/latest/get-started/>
- Curry, C. (2020). *Beartype*. `beartype`. <https://github.com/beartype/beartype>
- Dask Development Team. (2016). *Dask: Library for dynamic task scheduling*. <https://docs.dask.org>
- Hagenaars, R. (2019). *Nptyping*. <https://github.com/ramonhagenaars/nptyping>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Mendoza, M., & Pradeep Kumar Srinivasan. (2020, September 16). *PEP 646 – Variadic Generics*. Python Enhancement Proposals; Python Software Foundation. <https://peps.python.org/pep-0646/>

- 164 Miles, A., Kirkham, J., Durant, M., Bourbeau, J., Onalan, T., Hamman, J., Zain Patel,
165 Shikharsg, Rocklin, M., Dussin, R., Schut, V., Andrade, E. S. D., Abernathey, R., Noyes,
166 C., Sbalmer, Pyup.io Bot, Tran, T., Saalfeld, S., Swaney, J., ... Banihirwe, A. (2020).
167 *Zarr-developers/zarr-python: V2.4.0* (Version v2.4.0). Zenodo. [https://doi.org/10.5281/
168 ZENODO.3773450](https://doi.org/10.5281/ZENODO.3773450)
- 169 Moxon, S. A. T., Solbrig, H., Harris, N. L., Kalita, P., Miller, M. A., Patil, S., Schaper, K.,
170 Bizon, C., Caufield, J. H., Cuesta, S. C., Cox, C., Dekervel, F., Dooley, D. M., Duncan,
171 W. D., Fliss, T., Gehrke, S., Graefe, A. S. L., Hegde, H., Ireland, A. J., ... Mungall,
172 C. J. (2026). LinkML: An open data modeling framework. *GigaScience*, 15, g1af152.
173 <https://doi.org/10.1093/gigascience/giaf152>
- 174 Neurodata Without Borders. (2024). *Lindi*. Neurodata Without Borders. [https://github.com/
175 NeurodataWithoutBorders/lindi](https://github.com/NeurodataWithoutBorders/lindi)
- 176 numpy. (2017). *Numpy-stubs*. <https://github.com/numpy/numpy-stubs>
- 177 numpy. (2025). *Numtype*. NumPy. <https://github.com/numpy/numtype>
- 178 *NumPy 1.20.0 Release Notes*. (2021, January 30). NumPy Manual. [https://numpy.org/
179 devdocs/release/1.20.0-notes.html](https://numpy.org/devdocs/release/1.20.0-notes.html)
- 180 Python Software Foundation. (n.d.). *Typing PEPs*. Python Enhancement Proposals (PEPs).
181 Retrieved June 10, 2026, from <https://peps.python.org/topic/typing/>
- 182 Rübel, O., Tritt, A., Ly, R., Dichter, B. K., Ghosh, S., Niu, L., Baker, P., Soltesz, I., Ng,
183 L., Svoboda, K., Frank, L., & Bouchard, K. E. (2022). The Neurodata Without Borders
184 ecosystem for neurophysiological data science. *eLife*, 11, e78362. [https://doi.org/10.7554/
185 eLife.78362](https://doi.org/10.7554/eLife.78362)
- 186 Smith, E. V. (2017, June 2). *PEP 557 – Data Classes*. Python Enhancement Proposals
187 (PEPs). <https://peps.python.org/pep-0557/>
- 188 Tritt, A. J., Rübel, O., Dichter, B., Ly, R., Kang, D., Chang, E. F., Frank, L. M., &
189 Bouchard, K. (2019). HDMF: Hierarchical Data Modeling Framework for Modern Science
190 Data Standards. *2019 IEEE International Conference on Big Data (Big Data)*, 165–179.
191 <https://doi.org/10.1109/BigData47090.2019.9005648>
- 192 van Rossum, G., Lehtosalo, J., & Langa, Ł. (2014, September 29). *PEP 484 – Type Hints*.
193 Python Enhancement Proposals (PEPs). <https://peps.python.org/pep-0484/>
- 194 Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A.,
195 Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes,
196 A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers,
197 R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data management and
198 stewardship. *Scientific Data*, 3(1), 160018. <https://doi.org/10.1038/sdata.2016.18>